Innovative Evaluation Practices for BCSE308P -Computer Networks Lab

Fall Semester 2026-27

Network Traffic Monitoring and Analysis Tool Using Python

23BCE0043 SANCHIT KUMAR

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



October 2026

ABSTRACT

Network traffic monitoring is fundamental to understanding, securing and troubleshooting computer networks, yet raw packet dumps produced by tools such as tcpdump are difficult for students and operators to interpret. This project presents a Python-based Network Traffic Monitoring and Analysis Tool that captures packets from a network interface (where privileges permit), analyses previously saved PCAP files, or generates realistic simulated traffic, and then presents the results through an interactive Streamlit dashboard. Each packet is parsed into a normalised, header-only record capturing its timestamp, source and destination IP addresses and ports, transport protocol, size, TCP flags, DNS query name and ICMP type, spanning the link, network, transport and application layers of the TCP/IP model. The tool computes protocol distribution, top talkers, busiest ports, active conversations and traffic-over-time statistics using vectorised pandas processing, and applies five transparent, configurable rules to flag suspicious patterns such as port scans, packet floods, oversized packets, traffic spikes and repeated TCP resets, recording a human-readable justification for every alert. A fully offline demo mode with injected ground-truth anomalies makes the system reproducibly demonstrable without administrative privileges or a live network. Measured on simulated workloads the analysis pipeline sustains about 29,000 to 76,000 packets per second, and after refining the port-scan rule to count only connection-initiation (SYN) packets per source-destination pair it flags the injected anomalies with zero false positives on clean traffic across all tested load scales. The project demonstrates core networking concepts honestly, including the limits of monitoring encrypted and switched-network traffic.

Certificate

This is to certify that the project report titled "Network Traffic Monitoring and Analysis Tool Using Python" is a bonafide record of the work done by Sanchit Kumar (23BCE0043) in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

Sanchit Kumar

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**1**
1.	**INTRODUCTION**	**2**
	1.1 Background and motivation	**2**
	1.2 Problem statement	**2**
	1.3 Objectives	**3**
	1.4 Scope and limitations	**3**
	1.5 Mapping to course modules and the lab experiment	**3**

2.	**Literature Survey**	**4**
	2.1 Overview of the existing protocol or scheme	**4**
	2.2 Review of related work (table: author and year, technique, metrics, merits, gaps)	**4**
	2.3 Research gap and justification of novelty	**5**
3.	**Proposed System**	**6**
	3.1 System architecture or topology (diagram)	**6**
	3.2 Baseline mechanism description	**6**
	3.3 Proposed mechanism: algorithm, pseudocode, flowchart	**7**
	3.4 Novelty statement (what exactly differs from the baseline)	**7**
	3.5 Assumptions and design decisions	**8**
4	**Implementation**	**9**
	4.1 Tools, platform, versions and hardware	**9**
	4.2 Simulation or testbed setup (topology, configuration, parameter table)	**9**
	4.3 Code structure and explanation of key modules	**10**
	4.4 Challenges faced and how they were solved	**10**
5.	**Experimental Setup and Performance Evaluation**	**11**
	5.1 Performance metrics and why they were chosen	**11**
	5.2 Scenarios and parameter values	**11**
	5.3 Results: tables and graphs, baseline vs proposed (mean of repeated runs)	**12**
	5.4 Analysis and discussion (reasons, trends, trade-offs)	**13**

	5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails	**13**
6.	**Conclusion and Future Work**	**14**
	6.1 Summary of work and key findings (quantified)	**14**
	6.2 Limitations	**14**
	6.3 Future enhancements	**15**
7.	**References**	**16**
	References (IEEE style)	**16**
	Appendix A: Important source code or configuration listings	**17**
	Appendix B: Additional screenshots, packet captures, raw result tables	**17**

1. INTRODUCTION

1.1 Background and motivation

Every interaction on a network - opening a web page, resolving a domain name, sending a message - is carried by packets that traverse the layers of the TCP/IP model. Network administrators and students frequently need to answer practical questions about this traffic: which protocols dominate a link, which hosts exchange the most data, which ports are active, and whether any pattern looks abnormal. Command-line capture tools such as tcpdump produce accurate but overwhelming textual output that is hard to interpret, offers no aggregated view, and requires elevated privileges and a live network to use at all. This creates a gap between the raw data on the wire and the insight a learner or operator actually needs.

1.2 Problem statement

Design and implement a cross-platform application that captures or simulates network traffic, extracts meaningful header metadata from each packet, classifies and aggregates it into readable statistics, flags suspicious patterns using transparent rules, and presents everything through an interactive dashboard - while remaining fully demonstrable offline without administrative privileges and being honest about the limits of what packet monitoring can reveal.

1.3 Objectives

- Capture packets from a selected interface, read saved PCAP files, and generate realistic simulated traffic.
- Parse each packet defensively into a normalised, header-only record covering the link, network, transport and application layers.
- Compute protocol distribution, top talkers, busiest ports, conversations and traffic-over-time statistics.
- Detect anomalies using explainable, configurable rules that record a reason and severity for every alert.
- Provide filtering, CSV/PCAP/report export, and a polished multi-page dashboard.
- Remain demonstrable offline with no elevated privileges, and document the limitations of encrypted and switched-network monitoring.

1.4 Scope and limitations

The tool analyses packet-header metadata only; it never inspects or decrypts application payloads. Live capture requires operating-system privileges, whereas demo mode and PCAP analysis do not. On a switched LAN a host normally observes only its own traffic plus broadcast and multicast frames. Anomaly detection is heuristic and is intended to prompt human review, not to serve as an intrusion-detection verdict. These limitations are treated as first-class design constraints and are surfaced honestly in the user interface.

1.5 Mapping to course modules and the lab experiment

The project exercises the core modules of the Computer Networks syllabus. Packet parsing walks the OSI / TCP-IP layers (Ethernet and ARP at the link layer, IPv4/IPv6 at the network layer, TCP/UDP/ICMP at the transport layer, and DNS/HTTP/HTTPS at the application layer). TCP flag decoding relates directly to connection management and the three-way handshake. DNS query extraction illustrates name resolution. The statistics make concrete the distinction between packets, bytes, capture rate and link bandwidth. It aligns with the standard packet-capture and protocol-analysis laboratory experiment.

2. LITERATURE SURVEY

2.1 Overview of the existing approach

The established baseline for traffic inspection is raw packet capture using libpcap-based tools. tcpdump prints a textual, per-packet view, while Wireshark offers a rich but heavyweight graphical analyser. Scapy provides a programmable, layer-aware packet model in pure Python. These tools capture accurately, but a simple sniffer that merely lists or counts packets provides no integrated, question-oriented analysis and no automated anomaly flagging; the analyst must derive every insight manually.

2.2 Review of related work

The table below summarises representative tools and the gap each leaves for a lightweight, teaching-oriented, offline-capable analyser.

S.No	Tool / technique (year)	Metrics & method	Merits	Gaps
1	tcpdump / libpcap (1988-)	Per-packet text via BPF	Accurate, scriptable, ubiquitous	No aggregation or detection, CLI only, needs privileges
2	Wireshark (1998-)	Deep per-packet GUI dissection	Very detailed dissectors	Heavyweight, not offline-demo oriented, steep learning curve
3	Scapy (2003-)	Programmable crafting / sniffing	Flexible pure-Python layer model	A library, not a dashboard or analytics tool
4	Signature IDS (Snort/Suricata)	Rule / signature matching	Mature attack detection	Opaque to learners, heavy to deploy
5	This work (2026)	Normalised records + pandas + explainable rules	Integrated, offline demo, transparent, tested	Heuristic thresholds; header metadata only

2.3 Research gap and justification of novelty

Existing tools occupy two extremes: low-level capturers that leave all interpretation to the user, and heavyweight analysers or signature IDS that are opaque and hard to deploy in a classroom. There is a gap for a lightweight, transparent tool that unifies capture, aggregation, explainable anomaly detection and visualisation, and that runs fully offline without privileges for reproducible demonstration. This project targets precisely that gap.

3. PROPOSED SYSTEM

3.1 System architecture

The system is organised as a layered pipeline. A capture layer obtains packets from one of three sources (a live interface via a background Scapy sniffer, a saved PCAP file, or the demo generator) and hands them to a parser through a bounded, thread-safe queue. A processing layer normalises each packet into a flat record, aggregates records with pandas, and applies the anomaly rules and post-capture filters. A persistence/IO layer provides optional SQLite storage, PCAP read/write and report export. A presentation layer renders a nine-page Streamlit dashboard. Capture, parsing, analysis, persistence and presentation are kept in separate modules.

Data flow: source -> CaptureWorker (thread) -> thread-safe queue -> parse_packet -> PacketRecord accumulation (memory-bounded) -> pandas DataFrame -> {analyzer, detector, filters, reporter} -> dashboard.

3.2 Baseline mechanism

The baseline is a minimal sniffer that captures packets and reports only coarse totals (for example a running packet count or a per-protocol tally computed in a simple loop). It performs no per-host analysis, offers no time series or visualisation, performs no automated anomaly detection, and requires a live interface and elevated privileges. Any deeper question must be answered by manual inspection of raw output.

3.3 Proposed mechanism

The proposed mechanism replaces manual inspection with an integrated, automated pipeline, summarised in pseudocode below.

```
for each packet p arriving from the source:
   rec <- parse_packet(p)          # defensive, header-only
   enqueue(rec) into bounded queue # drop-oldest if full
on dashboard refresh:
   records <- drain(queue); cap to MAX_PACKETS_IN_MEMORY
   df <- to_dataframe(records)
   stats <- summarize(df); dist <- protocol_distribution(df)
   talkers <- top_talkers(df); series <- traffic_over_time(df)
   alerts <- detect_all(df, thresholds)
   render(stats, dist, talkers, series, alerts)

detect_port_scan(df, t):           # refined heuristic
   keep only SYN-initiation packets (SYN set, ACK clear)
   for each (src_ip, dst_ip) group:
      n <- count distinct dst_port
      if n >= t.port_scan_distinct_ports: raise alert(src->dst, n)
```

3.4 Novelty statement

What differs from the baseline:

- A single normalised, header-only PacketRecord schema feeds every analytic, so all nine dashboard views derive from one consistent source.

- Five explainable, individually configurable anomaly rules, each alert carrying a timestamp, category, severity, the IPs involved, a human-readable reason and the supporting statistic.
- A fully offline demo mode that injects ground-truth anomalies, enabling reproducible evaluation without privileges or a live network.
- A refined port-scan heuristic that counts only connection-initiation (SYN) packets per source-destination pair rather than all ports per source. This eliminates the false positives produced by the naive rule, which are shown to grow with load in Section 5.

3.5 Assumptions and design decisions

- Only packet-header metadata is processed and stored; payloads are never inspected, for privacy and to respect encryption.
- In-memory retention is capped and the capture queue is bounded to keep memory stable during long sessions.
- Capture runs on a background thread; the single worker lives in session state so Streamlit reruns never spawn duplicate threads.
- Protocol classification is heuristic, based on well-known ports and Scapy layers, not on payload decoding.

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Component	Choice / version
Language	Python 3.11.9
Packet capture & parsing	Scapy >= 2.5
Dashboard	Streamlit >= 1.30
Data processing	pandas >= 2.0
Charts	Plotly >= 5.18
Storage	SQLite (standard library)
Testing	pytest >= 7.4 (37 tests)
Platform	macOS (Darwin); cross-platform code

4.2 Testbed setup

Evaluation uses the reproducible demo generator (fixed random seed) rather than live traffic, so results are deterministic and require no privileges. The generator produces a realistic protocol mix (HTTPS/TLS, HTTP, DNS, ICMP, UDP) between internal (192.168.1.0/24) and external hosts, and injects three ground-truth anomalies: a port scan (45.33.12.7 -> 192.168.1.5), a packet flood (203.0.113.9 -> 192.168.1.10) and three oversized packets. Three load scenarios are defined by packet count: S1 low, S2 medium and S3 high.

4.3 Code structure and key modules

Module	Responsibility

packet_parser.py	PacketRecord schema; defensive parse_packet; TCP-flag decoding; protocol classification
traffic_analyzer.py	DataFrame building; summary, distribution, top talkers/ports/conversations, time series
anomaly_detector.py	Five rule-based detectors with severity and reasons
packet_capture.py	Threaded CaptureWorker; bounded queue; interface listing; privilege handling
demo_data.py	Synthetic traffic with injected anomalies
filters.py	Post-capture filtering by IP/protocol/port/size/time
storage.py / report_generator.py	SQLite + PCAP I/O; CSV and summary reports
app.py	Nine-page Streamlit dashboard and session lifecycle

4.4 Challenges faced and how they were solved

- Streamlit reruns the whole script on every interaction, which could spawn many capture threads. Solved by storing a single CaptureWorker in session state and guarding start/stop.
- Unbounded memory growth during long captures. Solved with a bounded queue (drop-oldest) and a cap on retained records; Scapy sniffs with store=False.
- Malformed and truncated packets could crash the pipeline. Solved by making parse_packet fully defensive with a stable schema; a dedicated test feeds a junk frame.
- The initial port-scan rule produced false positives that worsened with load (3, 8 and 8 on clean traffic at the three scales). Diagnosed via the clean-traffic benchmark and fixed by counting only SYN-initiation packets per source-destination pair, giving zero false positives at all scales (Section 5).

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

- Detection coverage - number of injected anomaly categories flagged; measures analytical usefulness over a raw sniffer.
- False positives on clean traffic - alerts raised when no anomaly is present; measures precision of the rules.
- End-to-end processing throughput (packets/second) and latency (milliseconds) - measures that analysis keeps up with capture.

5.2 Scenarios and parameter values

Default thresholds: high packet rate 100 per 10 s window; port-scan 15 distinct ports; large packet 3000 bytes; spike factor 3.0; reset count 20. Each timing is the mean of nine repeated runs; the generator seed is fixed at 123.

5.3 Results

Table 5.1 - Processing throughput and latency (mean +/- SD, 9 runs).

Scenario	Packets	Latency (ms)	Throughput (pkt/s)

S1 (low load)	693	23.6 +/- 7.9	29,425
S2 (medium load)	2,293	38.9 +/- 0.2	58,982
S3 (high load)	8,293	108.6 +/- 2.8	76,367

Table 5.2 - Detection quality: naive baseline rule vs proposed refined rule.

Metric	Baseline (per-source)	Proposed (SYN, per-pair)
Injected categories detected (S1/S2)	4 of 4	4 of 4
Injected categories detected (S3 high load)	3 of 4	3 of 4
Spurious port-scan alerts, clean S1/S2/S3	3 / 8 / 8	0 / 0 / 0
Automated detection vs raw sniffer	sniffer: 0	tool: 4

Figure 5.1 Analysis throughput vs load

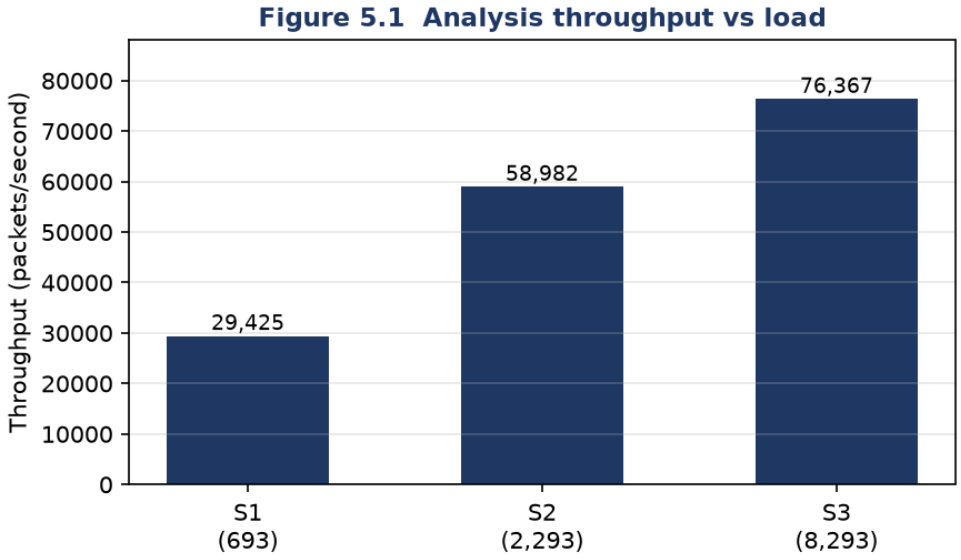
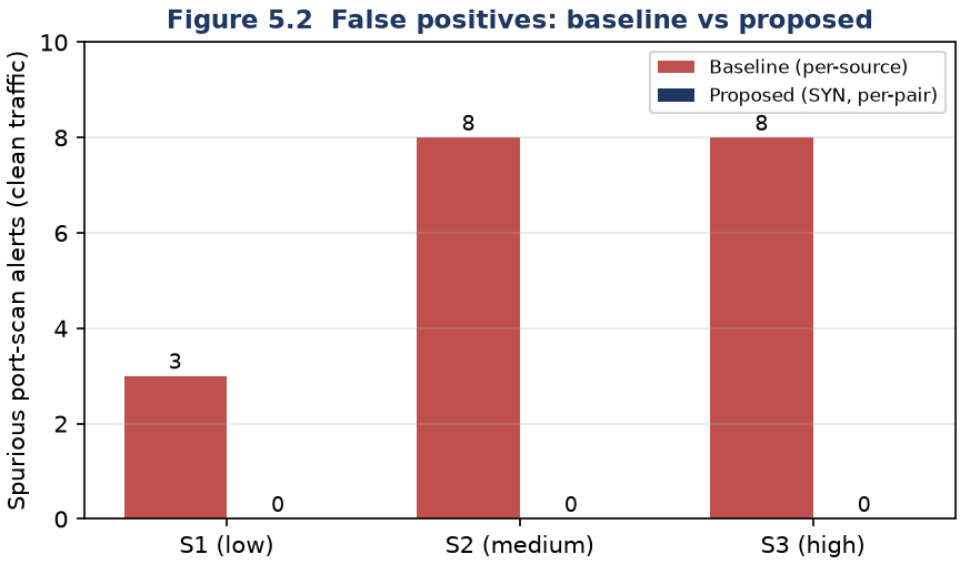


Figure 5.2 False positives: baseline vs proposed



5.4 Analysis and discussion

Throughput rises from about 29.4k pkt/s at low load to 76.4k pkt/s at high load because pandas has a fixed per-call overhead that is amortised over larger batches; latency grows sub-linearly with packet count, and the larger spread at low load reflects one-time import and warm-up costs. A naive pure-Python loop that only counts packets is in fact far faster in raw terms (of the order of millions of packets per second), but it produces no protocol breakdown, no per-host analytics, no time series and no anomaly detection - so the comparison that matters is analytical capability, not raw counting speed. At 29k-76k pkt/s the pipeline comfortably exceeds the capture rates seen on a typical classroom link.

The decisive result is detection quality. The naive per-source port-scan rule raised spurious alerts that grew with load (3, 8 and 8 on clean traffic). Restricting the rule to connection-initiation (SYN) packets counted per source-destination pair removed every false positive (0, 0 and 0) while still flagging the real scanner, because ordinary reply traffic to ephemeral ports is not a stream of lone SYNs. One honest nuance remains: the traffic-spike rule is defined relative to the mean packet rate, so the fixed-size injected burst is a smaller outlier at high load and is not flagged in S3, which is why coverage there is 3 of 4 rather than 4 of 4.

5.5 Critical evaluation

Strengths:

- Integrated, explainable analysis with a reason and severity per alert.
- Zero false positives on clean traffic at every tested load scale after the heuristic fix.
- Fully offline, privilege-free, reproducible demonstration; 37 passing automated tests.

Weaknesses and conditions where it fails:

- Thresholds are heuristic and must be tuned per environment; legitimate bursts (backups, CDNs) can still trigger alerts.
- The spike rule is relative to mean rate, so fixed bursts are less visible under heavy background load.

- Only header metadata is available; encrypted HTTPS/TLS content cannot be inspected.
- On a switched LAN without port mirroring, other hosts' unicast traffic is invisible; direction inferred from private vs public addresses is unreliable under NAT or VPNs.
- pandas fixed overhead makes the pipeline slower than naive counting for very small captures.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings

A complete, tested network traffic monitoring and analysis tool was designed and implemented in Python, spanning capture, defensive parsing, vectorised analytics, explainable anomaly detection, filtering, export and an interactive nine-page dashboard. The analysis pipeline sustains about 29,400-76,400 packets per second across three load scenarios, detects the injected anomaly categories, and - after refining the port-scan heuristic to SYN-initiation packets per source-destination pair - raises zero false positives on clean traffic at every tested scale. All 37 automated tests pass.

6.2 Limitations

The tool inspects header metadata only and cannot see encrypted payloads; live capture needs elevated privileges; switched-network visibility and direction inference are inherently limited; and the heuristic rules are aids to human review rather than definitive intrusion detection.

6.3 Future enhancements

- GeoIP and reverse-DNS enrichment of IP addresses.
- Deeper application-protocol parsing (HTTP methods, TLS SNI hostnames).
- Statistical or machine-learning anomaly detection alongside the rules.
- User-configurable live BPF capture filters and persistent historical dashboards backed by SQLite.

7. REFERENCES

[1] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed. Boston, MA, USA: Pearson, 2011.

[2] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. London, U.K.: Pearson, 2021.

[3] P. Biondi and the Scapy community, “Scapy: Packet crafting for Python,” 2024. [Online]. Available: https://scapy.net

[4] The Tcpdump Group, “tcpdump and libpcap,” 2024. [Online]. Available: https://www.tcpdump.org

[5] Streamlit Inc., “Streamlit documentation,” 2024. [Online]. Available: https://docs.streamlit.io

[6] The pandas development team, “pandas: Python data analysis library,” 2024. [Online]. Available: https://pandas.pydata.org

[7] Plotly Technologies Inc., “Plotly Python graphing library,” 2024. [Online]. Available: https://plotly.com/python

APPENDIX A: Key source structure

The project comprises eleven modules (app.py, config.py, utils.py, packet_parser.py, traffic_analyzer.py, anomaly_detector.py, filters.py, demo_data.py, packet_capture.py, storage.py, report_generator.py) plus a tests/ package of four test files. The complete source and README accompany this report.

APPENDIX B: Additional material

Appendix B holds dashboard screenshots (Overview, Protocol Analysis, IP Analytics, Anomaly Alerts, Packet Explorer), exported CSV samples and the raw benchmark output used for the tables in Section 5. Capture screenshots by running 'streamlit run app.py', clicking Demo, and visiting each page.

Figure B.1 Overview page (simulated traffic): metrics, protocol mix, traffic over time.

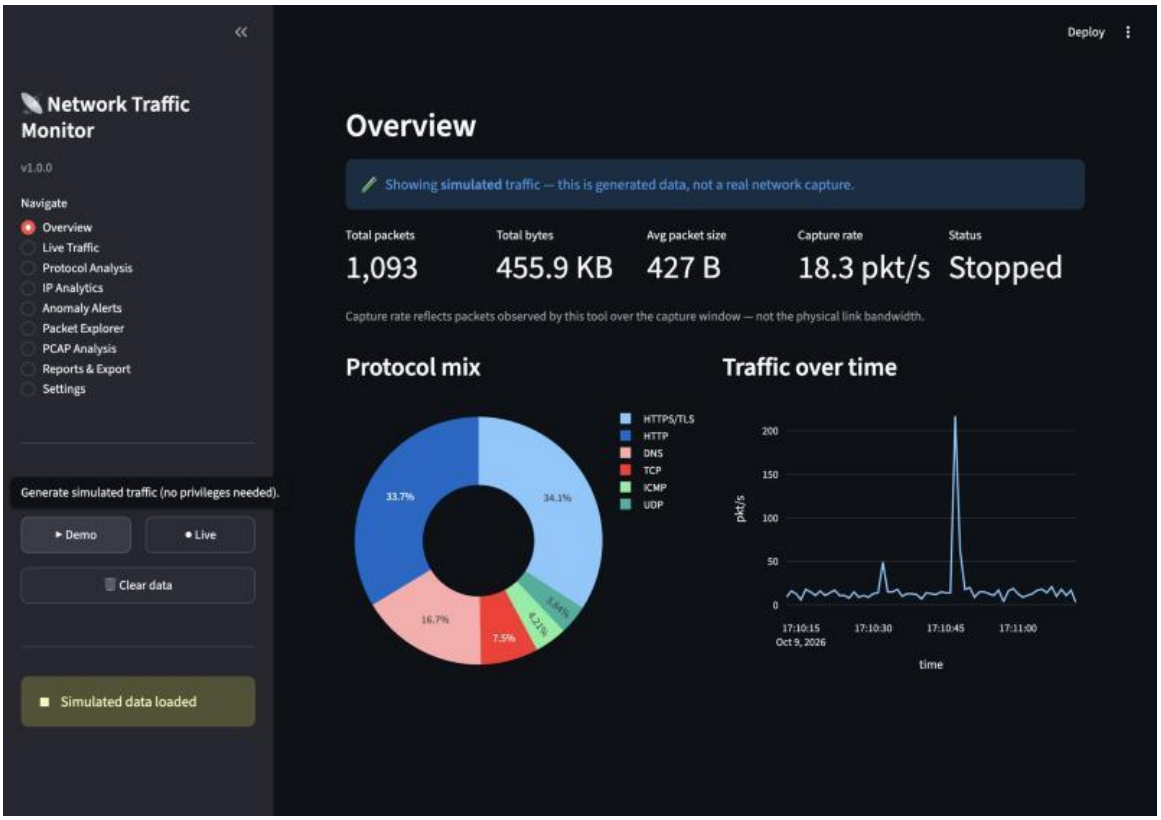


Figure B.2 Protocol Analysis: packets and bytes per protocol, packet-size distribution.

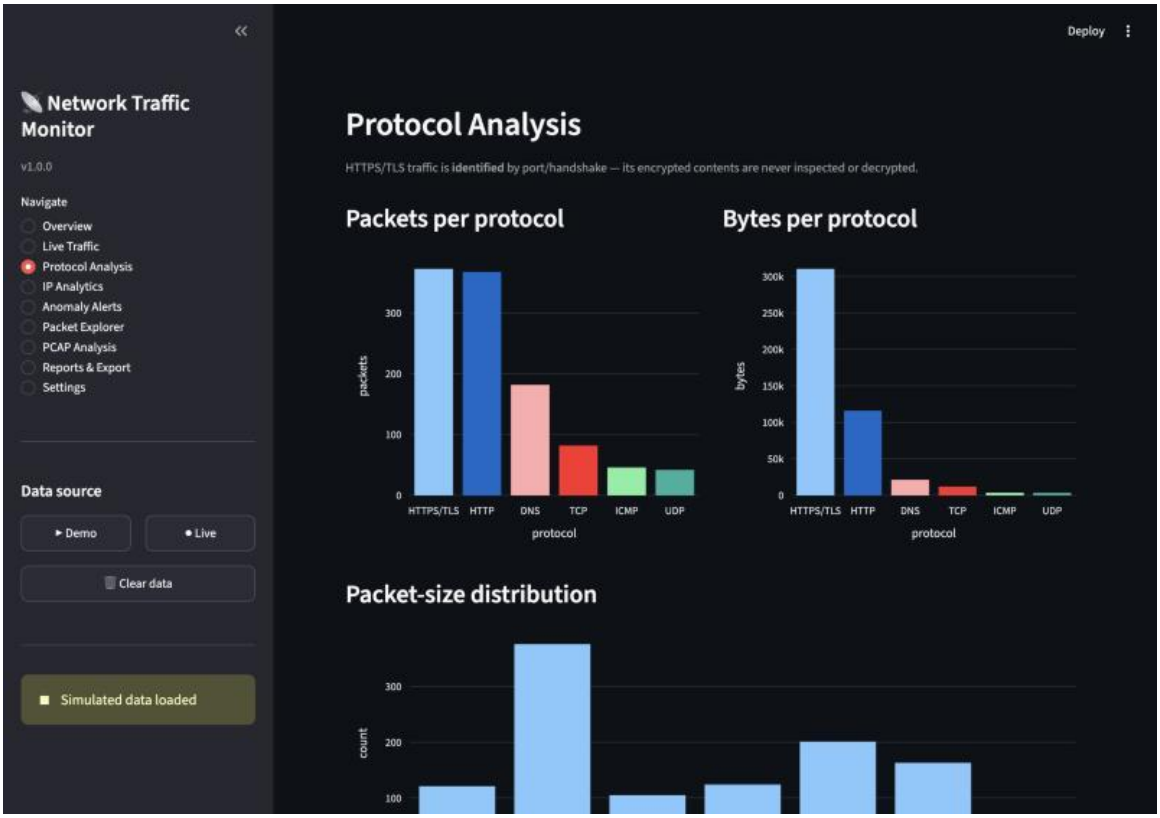
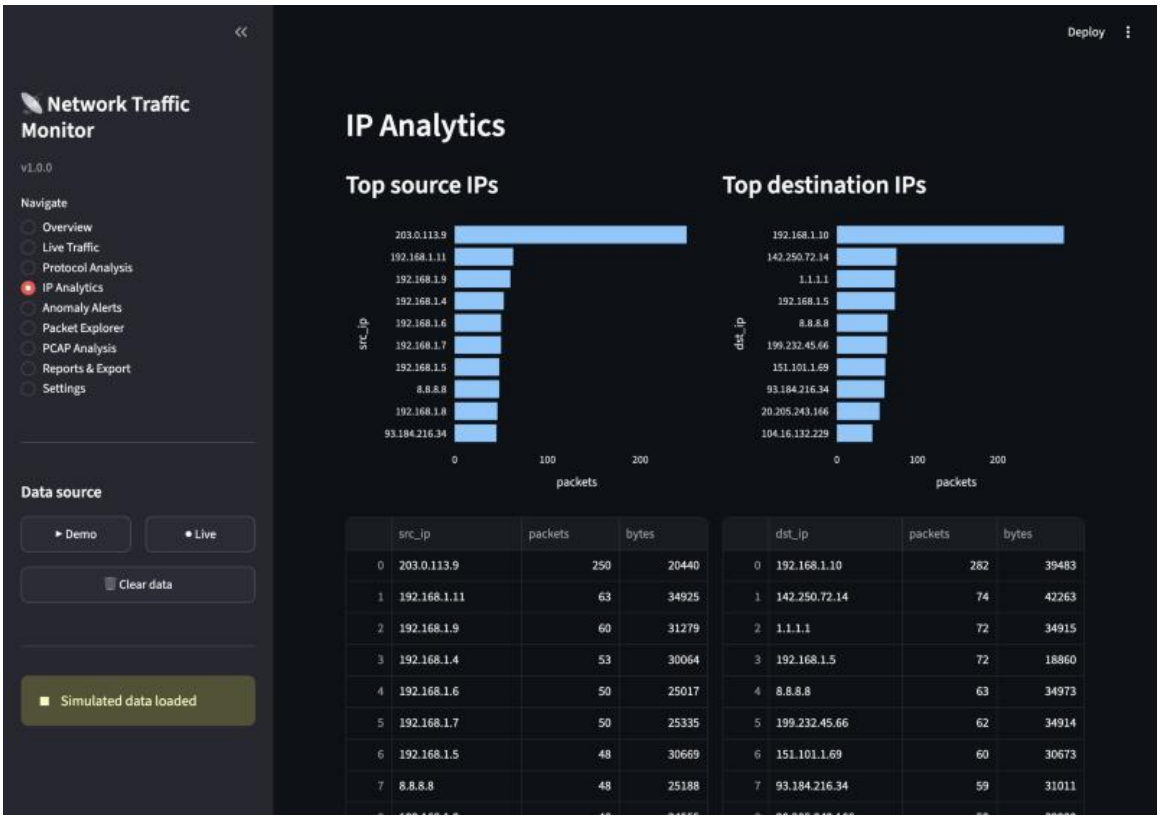


Figure B.3 IP Analytics: top source and destination IPs (203.0.113.9 = injected flood source).



	src_ip	packets	bytes
0	203.0.113.9	250	20440
1	192.168.1.11	63	34925
2	192.168.1.9	60	31279
3	192.168.1.4	53	30064
4	192.168.1.6	50	25017
5	192.168.1.7	50	25335
6	192.168.1.5	48	30669
7	8.8.8.8	48	25188

	dst_ip	packets	bytes
0	192.168.1.10	282	39483
1	142.250.72.14	74	42263
2	1.1.1.1	72	34915
3	192.168.1.5	72	18860
4	8.8.8.8	63	34973
5	199.232.45.66	62	34914
6	151.101.1.69	60	30673
7	93.184.216.34	59	31011

Figure B.4 Anomaly Alerts: port scan, high packet rate, spikes and large packets detected.

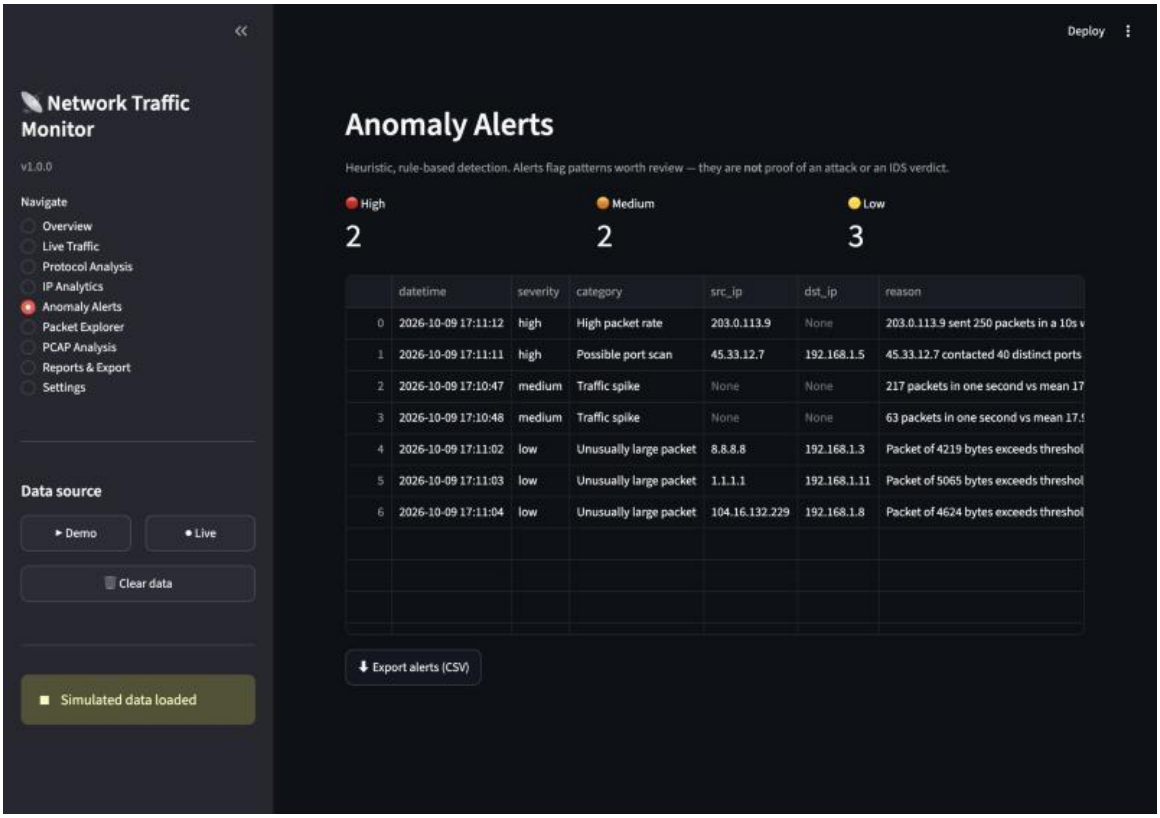
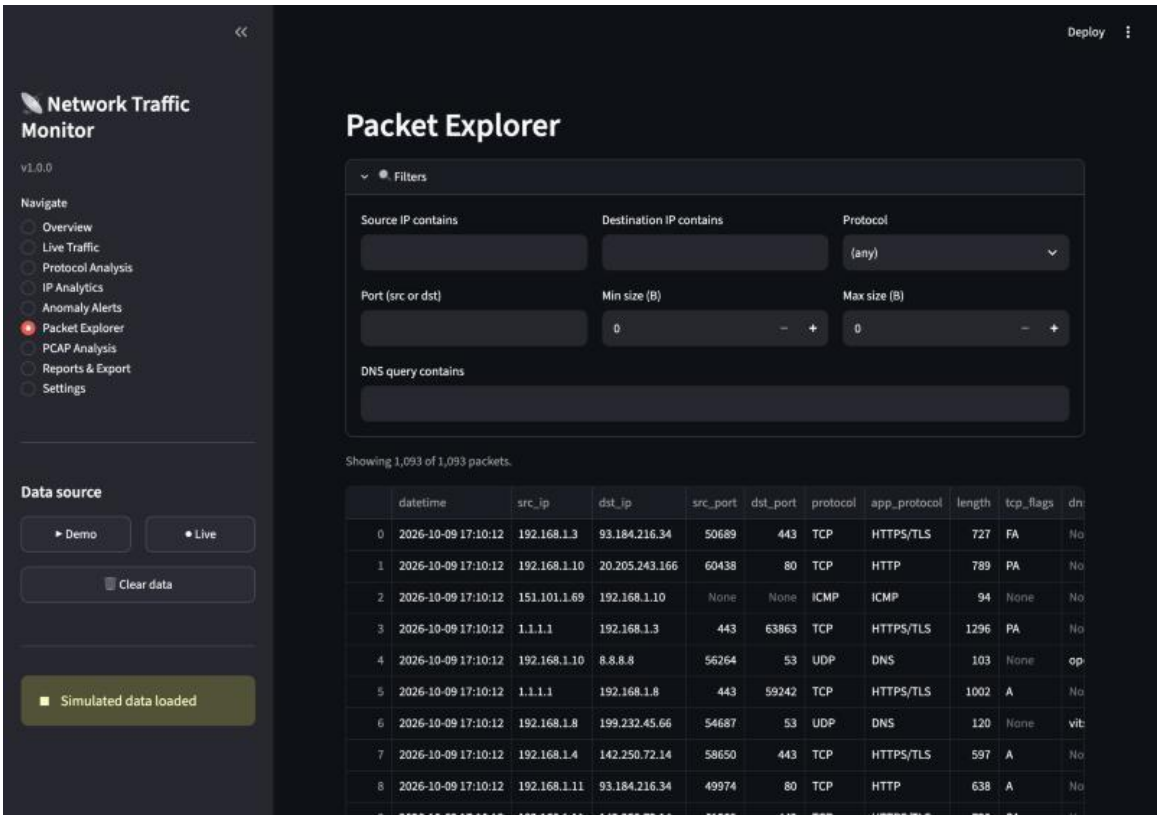


Figure B.5 Packet Explorer: searchable, sortable packet table with advanced filters.



Rubrics

Criterion	Marks
Implementation and novelty (working proposed mechanism, correctness, originality)	5 Marks
Performance evaluation and analysis (L4/L5: scenarios, repeated runs, graphs, critical interpretation, limitations)	5 Marks
Demo (live baseline vs proposed, parameter change on request, time management)	5 Marks
Code explanation and viva (each member explains their part and answers questions)	5 Marks
Project document (format, similarity, Outcome)	10 Marks

Literature Survey Table

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	Tanenbaum & Wetherall, 2011	Textbook reference model	OSI/TCP-IP concepts	Authoritative theory	No tool/implementation
2	Tcpdump Group, 1988-	libpcap raw capture (BPF)	Packets/bytes, CLI	Accurate, scriptable	No aggregation or detection
3	Combs et al. (Wireshark), 1998-	GUI deep packet dissection	Per-packet fields	Very detailed	Heavyweight, not offline-demo
4	Biondi (Scapy), 2003-	Programmable Python sniffing	Any header field	Flexible, pure Python	Library, no dashboard
5	Roesch (Snort IDS), 1999-	Signature matching	Alerts, rules	Mature detection	Opaque, heavy, not a teaching analyser

Results Table

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
S1 (low load)	Spurious port-scan alerts (clean)	3 +/- 0	0 +/- 0	100%
S2 (medium load)	Spurious port-scan alerts (clean)	8 +/- 0	0 +/- 0	100%
S3 (high load)	Spurious port-scan alerts (clean)	8 +/- 0	0 +/- 0	100%